

AL-AZHARI LIBRARY — Phase 6: Cart Implementation Report

Phase 6 complete. Implementation stopped here. No Phase 7 Inventory work was implemented.

1. Executive Summary

Phase 6 implements the **Cart Domain** for the AL-AZHARI LIBRARY backend in strict conformance with the authoritative specifications:

- AL-AZHARI-LIBRARY-BACKEND-IMPLEMENTATION-PLAN.pdf (Sections 20, 48.3, 62, 63)
- AL-AZHARI-LIBRARY-MongoDB-Implementation-Plan.pdf (Sections 5.6, 32.3, 34)
- AL-AZHARI-LIBRARY-PRD.pdf (CART-001 through CART-005)

Core Architectural Guarantees

- Display Snapshot Only:** The price stored in the cart (`unitPriceMinor`) is strictly a display snapshot. It is NOT authoritative financial data. Cart is NOT an order and does NOT reserve stock.
- No Inventory Mutation:** Cart operations never mutate `stockTotal` , `stockReserved` , or create reservation records. Inventory reservation is deferred to Phase 7.
- Service Exclusion:** Custom student services are strictly rejected at the cart boundary; only fixed-price products/variants enter the product cart.
- Optimistic Versioning:** Every cart mutation atomically checks and increments the cart `version` . Stale writes abort with HTTP 409 `ConflictError` (`CART_VERSION_CONFLICT`), eliminating race conditions and last-write-wins overwrites.
- Never Silently Discard Conflicts:** Incompatible items, changed prices, deleted variants, or out-of-stock items encountered during guest-to-user merge return a structured conflict list requiring customer review, leaving the guest cart intact.

2. Files Created & Modified

Created Files

- `cart.types.ts`: Data contracts for `CartOwnerContext` , `ICart` , `ICartItem` , `CartResponse` , `CartMergeConflict` , and `CartMergeResult` .
- `cart.model.ts`: Strict Mongoose schema with `_id: true` subdocuments, integer validators, and blueprint indexes.
- `cart.repository.ts`: Data access layer with application-level TTL checks, owner queries, and atomic optimistic version predicate updates.
- `cart.service.ts`: Business logic implementing catalog re-validation, variant eligibility, snapshot computation, and deterministic merge matrix.
- `cart.controller.ts`: REST controller implementing standard envelopes, status codes, and error delegation.
- `cart.routes.ts`: Express routing pipeline combining optional authentication, cart owner resolution, and Zod request validation.
- `cart.schemas.ts`: Strict Zod validation schemas for `addItem` , `updateItem` , `removeItem` , `mergeCart` , and `itemIdParam` .
- `cart-owner.middleware.ts`: Resolves authenticated principal (`ownerType: 'user'`) or generates/validates unpredictable guest session identifiers (`ownerType: 'guest'`) via cookies and headers.
- `20260928_004_cart.migration.ts`: Idempotent, non-destructive migration script creating `carts` blueprint indexes.
- `PHASE-6-CART.md`: Comprehensive Phase 6 technical documentation.
- `docs/PHASE-6-CART.md`: Root workspace documentation reference.

Modified Files

- `errorCodes.ts`: Added Cart error constants (`CART_NOT_FOUND` , `CART_VERSION_CONFLICT` , `CART_ITEM_NOT_FOUND` , `PRODUCT_NOT_PURCHASABLE` , `VARIANT_REQUIRED` , `VARIANT_NOT_FOUND` , `VARIANT_NOT_PURCHASABLE` , `INVALID_QUANTITY` , `CART_OWNERSHIP_DENIED` , `CART_MERGE_CONFLICT`).
- `AppError.ts`: Extended `NotFoundError`, `ForbiddenError`, and `ValidationError` constructors to support domain error codes.
- `app.ts`: Mounted `cartRouter` under `/api/v1/cart` .
- `index.ts` (migrations): Exported `cartMigration`.
- `index.ts` (carts): Exported all Cart types, models, repositories, services, controllers, and routes.

3. API Contract & Implementation

Mounted under `/api/v1/cart` :

Endpoint	Method	Authentication	Request Body / Query	Success Response	Error Codes
<code>/</code>	GET	Guest session or Customer	None	200 OK (Cart representation with computed totals)	500
<code>/items</code>	POST	Guest session or Customer	{ <code>productId</code> , <code>variantId?</code> , <code>quantity</code> , <code>expectedVersion?</code> }	200 OK (Updated cart)	400, 404, 409, 422
<code>/items/:itemId</code>	PATCH	Cart Owner	{ <code>quantity</code> , <code>expectedVersion</code> }	200 OK (Updated cart)	400, 404, 409, 422
<code>/items/:itemId</code>	DELETE	Cart Owner	Query/Body { <code>expectedVersion?</code> }	204 No Content	404, 409
<code>/merge</code>	POST	Customer (<code>requireAuthentication</code>)	{ <code>sessionId?</code> , <code>expectedUserCartVersion?</code> }	200 OK ({ <code>cart</code> , <code>conflicts: []</code> })	401, 409

4. Database Schema & Blueprint Indexes

carts Collection Schema

```
_id: ObjectId
ownerType: 'guest' | 'user'
userId: ObjectId | null
sessionId: string | null
items[]:
  _id: ObjectId
  productId: ObjectId (ref: Product)
  variantId: string | null
  quantity: integer >= 1
  unitPriceMinor: integer >= 0 (piastres snapshot)
  productNameSnapshot: { ar: string, en?: string | null }
  imageSnapshot: string | null
  addedAt: UTC Date
currency: 'EGP'
expiresAt: UTC Date | null (30 days for guest, null for user)
version: integer >= 1 (optimistic locking counter)
createdAt: UTC Date
updatedAt: UTC Date
```

MongoDB Indexes Defined

- 1. idx_carts_user_unique : { userId: 1 } (unique, partialFilterExpression: { ownerType: "user" })
- 2. idx_carts_session_unique : { sessionId: 1 } (unique, partialFilterExpression: { ownerType: "guest" })
- 3. idx_carts_expires_ttl : { expiresAt: 1 } (expireAfterSeconds: 0)

5. Merge Behavior & Conflict Matrix

Executed when an unauthenticated guest logs in and merges their guest cart into their registered account:

Scenario	Guest Cart	User Cart	Resolved Outcome
Same item in both	Product A (qty: 2)	Product A (qty: 1)	Combined: Product A (qty: 3)
Guest-only item	Product B (qty: 1)	—	Moved to user cart: Product B (qty: 1)
User-only item	—	Product C (qty: 3)	Preserved: Product C (qty: 3)
Different variants	Variant 1 (qty: 1)	Variant 2 (qty: 2)	Distinct line items preserved
Product unavailable	Out of stock	—	PRODUCT_UNAVAILABLE conflict
Price changed	Snapshot 185 EGP	Catalog 200 EGP	PRICE_CHANGED conflict
Variant deleted	Variant deleted	—	VARIANT_NOT_FOUND conflict
Product unpublished	Unpublished book	—	PRODUCT_UNPUBLISHED conflict

Zero Silent Discard Policy: If any guest item conflicts with current catalog rules or has drifted in price:

- 1. No item is deleted or silently replaced.
- 2. No quantities are silently changed.
- 3. The guest cart remains intact in MongoDB.

4. HTTP 409 `CART_MERGE_CONFLICT` is returned with the structured list of conflicts for customer review.

6. Test Suite & Verification Results

Final Verification Command Output

```
> al-azhari-library-backend@1.0.0 typecheck
> tsc --noEmit
# Result: PASS (0 errors)

> al-azhari-library-backend@1.0.0 lint
> eslint .
# Result: PASS (0 errors, 0 warnings)

> al-azhari-library-backend@1.0.0 build
> tsc
# Result: PASS (dist/ created cleanly)

> al-azhari-library-backend@1.0.0 test
> jest
```

Full Test Breakdown

- **Phase 5 Baseline:** 49 suites, 385 tests, 0 failures.
- **Phase 6 Delivery:**
 - `cart-schema.test.ts`: 7 tests
 - `cart-migration.test.ts`: 2 tests
 - `cart-service.test.ts`: 10 tests
 - `cart-guest.test.ts`: 5 tests
 - `cart-user.test.ts`: 2 tests
 - `cart-concurrency.test.ts`: 1 test
 - `cart-merge.test.ts`: 2 tests
 - `cart-security.test.ts`: 5 tests
 - `cart-smoke.test.ts`: 7 tests
- **Total Phase 6 Test Count:** 58 test suites passed, 429 tests passed, 0 failures.
- **Zero Regressions:** All 385 pre-existing tests across Auth, Users, Addresses, Categories, Products, Search, Content, and Database Infrastructure continue to pass with 100% success.

7. Explicitly Deferred Features

In strict accordance with the Phase 6 boundary:

- **Inventory Reservations & Deductions:** Phase 7
- **Checkout & Final Order Price Revalidation:** Phase 8
- **Orders & Status Machine:** Phase 8
- **Payments & Payment Proofs:** Phase 9
- **Shipping Rules & Coupons:** Phase 10
- **Student Services Workflows:** Phase 11
- **Customer Reviews & Ratings:** Phase 12

Phase 6 complete. Implementation stopped here. No Phase 7 Inventory work was implemented.